

TECHNICAL CASE STUDY

LIMIT OVERRUN DEMO VISUAL

ANÁLISIS DE CONCURRENCIA EN E-COMMERCE

EL ESCENARIO DE NEGOCIO

Contexto

Una tienda online lanza una promoción: "Cupón de \$50 USD de regalo".

Regla: Un solo uso por cuenta de usuario.

El Objetivo del Atacante

Aplicar el mismo cupón múltiples veces en un **mismo milisegundo** para acumular descuentos ilegítimos (Ej. \$500 en lugar de \$50).

EL FALLO EN EL BACKEND

Analizamos el código que procesa la petición. Observa el patrón **Check-then-Act**:

```
// POST /api/v1/apply-coupon
def apply_coupon(user_id, coupon_id):
    used = db.query("SELECT is_used FROM coupons WHERE id=?", coupon_id)
    if not used:
        # VENTANA DE CARRERA (RACE WINDOW)
        apply_discount_to_cart(user_id, 50)
        db.execute("UPDATE coupons SET is_used=true WHERE id=?", coupon_id)
```

MECÁNICA TOCTOU

Time-of-Check to Time-of-Use. La vulnerabilidad reside en el intervalo entre la verificación y la acción.

PASO 1

CHECK

¿Está usado?
Respuesta: NO

VENTANA DE CARRERA

El atacante inyecta 20 peticiones aquí

PASO 2

USE

Aplicar descuento y
marcar como usado.

SIMULACIÓN (MS)

Tiempo (ms)	Hilo A (Petición 1)	Hilo B (Petición 2)	Estado en Base de Datos
0ms	SELECT is_used?	---	is_used:
1ms	---	SELECT is_used?	is_used:
2ms	APLICAR DESCUENTO (\$50)	APLICAR DESCUENTO (\$50)	is_used: false (pend.)
3ms	UPDATE is_used = true	---	is_used: true
4ms	---	UPDATE is_used = true	is_used: true

 **Resultado:** Dos peticiones validadas. Descuento total: **\$100 USD**.

SINGLE-PACKET ATTACK (HTTP/2)

En el pasado, la latencia de red (jitter) hacía difícil ganar la carrera. Las peticiones llegaban dispersas.

Hoy, usamos **HTTP/2 Single-packet Attack**. Enviamos múltiples "streams" en un solo paquete TCP.

Sincronización Total

Esto garantiza que todas las peticiones lleguen al servidor **exactamente en el mismo instante**, colapsando la lógica de hilos del backend.

MITIGACIÓN I: DB LOCKING

Forzar la linealidad mediante bloqueos a nivel de fila en la base de datos.

```
BEGIN TRANSACTION;
-- Bloquea la fila para cualquier otra lectura/escritura
SELECT is_used FROM coupons WHERE id = 123 FOR UPDATE;

IF is_used = false:
    UPDATE coupons SET is_used = true WHERE id = 123;
COMMIT;
```

Con FOR UPDATE, la segunda petición **debe esperar** a que la primera haga el COMMIT, invalidando la carrera.

MITIGACIÓN II: ATOMICIDAD

Eliminar el patrón de lectura/escritura separada. Delegar la validación al motor de DB.

Vulnerable

```
if (stock > 0):  
    update stock -= 1
```

Seguro (Atomic)

```
UPDATE products  
SET stock = stock - 1  
WHERE id = 1 AND stock > 0;
```

Si la condición AND stock > 0 no se cumple, el motor de DB no realiza la acción y devuelve 0 filas afectadas.

RESUMEN PARA EL AUDITOR

Qué buscar

Cualquier endpoint que modifique un estado único: cupones, likes, saldo, inventario limitado.

Cómo auditar

Simular carga concurrente extrema. Analizar si el resultado final (consistencia) coincide con las reglas de negocio.